

Práctica Nro. 10

Paradigmas de Lenguaje de Programación

Objetivo: poder reconocer las diferentes características que poseen los paradigmas de programación

Ejercicio 1: Un programa en un lenguaje procedural es una secuencia de instrucciones o comandos que se van ejecutando y producen cambios en las celdas de memoria, a través de las sentencias de asignación. ¿Qué es un programa escrito en un lenguaje funcional? y ¿Qué rol cumple la computadora?

Un programa funcional está compuesto por funciones y expresiones, no por secuencias de instrucciones que modifican celdas de memoria. El valor más importante es la función y los programas se construyen mediante la aplicación y composición de funciones.

La computadora cumple el rol de evaluar expresiones, las interpreta y ejecuta reduciéndolas hasta obtener un valor final.

Ejercicio 2: ¿Cómo se define el lugar donde se definen las funciones en un lenguaje funcional?

Las funciones se definen dentro del script o programa funcional mediante definiciones que indican su comportamiento. Por ejemplo:

cuadrado :: num -> num

*cuadrado x = x * x*

También puede inferirse automáticamente el tipo de la función.

Ejercicio 3: ¿Cuál es el concepto de variables en los lenguajes funcionales?

Las variables tienen el concepto de variable matemática, no de celda de memoria. Representan valores desconocidos y no cambian durante la evaluación.

Ejercicio 4: ¿Qué es una expresión en un lenguaje funcional? ¿Su valor de qué depende?

La expresión es la noción central de la programación funcional. Es un bloque de código que se evalúa para producir un valor sin efectos secundarios. El valor de una expresión depende únicamente de los valores de las sub-expresiones que la componen y del entorno donde se definen las variables.

Ejercicio 5: ¿Cuál es la forma de evaluación que utilizan los lenguajes funcionales?

Los lenguajes funcionales utilizan la evaluación de expresiones, reduciendo expresiones complejas a otras más simples hasta obtener un valor final. Toda expresión denota un valor. Pueden usar evaluación perezosa (lazy), donde las expresiones se evalúan sólo cuando su valor es necesario; o estricta, en donde las sub-expresiones se evalúan inmediatamente. El resultado es independiente del método.

Ejercicio 6: ¿Un lenguaje funcional es fuertemente tipado? ¿Qué tipos hay? ¿Por qué?

Sí, generalmente los lenguajes funcionales son fuertemente tipados para garantizar seguridad, corrección, optimización y razonamiento formal. Existen:

- Tipos básicos: entero, flotante, booleano, carácter, etc.
- Tipos derivados: pares, funciones, polimórficos (representados mediante letras griegas como β).

Esto permite detectar errores de tipos durante la compilación y garantizar consistencia en las operaciones.

Ejercicio 7: ¿Cómo definiría un programa escrito en POO?

Un programa orientado a objetos es un conjunto de objetos que interactúan mandándose mensajes, instanciados desde clases con variables y métodos, utilizando encapsulamiento, herencia y polimorfismo para resolver problemas.

Ejercicio 8: Diga cuáles son los elementos más importantes y hable sobre ellos en la programación orientada a objetos.

Los elementos principales son:

- Objetos: entidades con estado interno y comportamiento.
- Mensajes: solicitudes enviadas entre objetos para que ejecuten una acción.
- Métodos: programas asociados a un objeto que definen su comportamiento.
- Clases: tipos definidos por el usuario que especifican atributos y métodos.
- Instancias: objetos concretos creados a partir de una clase.
- Encapsulamiento: oculta el estado interno, accesible sólo por métodos.
- Herencia: permite a una clase derivar propiedades de otra.
- Polimorfismo: objetos de distintas clases que responden a un mismo mensaje.

Ejercicio 9: La posibilidad de ocultamiento y encapsulamiento para los objetos es el primer nivel de abstracción de la POO, ¿cuál es el segundo?

El segundo nivel de abstracción es la herencia, ya que permite definir nuevas clases a partir de otras reutilizando atributos y métodos.

Ejercicio 10: ¿Qué tipos de herencias hay?Cuál usa Smalltalk y C++

- Herencia simple: una clase deriva de una sola clase base (Smalltalk utiliza herencia simple).
- Herencia múltiple: una clase deriva de varias clases base (C++ utiliza herencia múltiple).

Ejercicio 11: En el paradigma lógico ¿Qué representa una variable? ¿y las constantes?

- Variables: elementos indeterminados que pueden sustituirse por otros valores constantes y permite definir hechos. Comienzan con mayúscula.
- Constantes: elementos determinados o fijos del dominio. Se escriben en minúscula o como números.

Ejercicio 12: ¿Cómo se escribe un programa en un lenguaje lógico?

Un programa lógico es una secuencia de cláusulas. Estas cláusulas pueden ser:

- Hechos: afirmaciones sobre relaciones que son verdaderas en el dominio. Se escriben como predicados con constantes o términos [*tiene(coche,ruedas)*].
- Reglas: relaciones lógicas que definen como se derivan nuevos hechos a partir de otros hechos o reglas, usando implicaciones [*conclusión :- condición*].
- Consultas: preguntas que el usuario plantea al sistema para obtener soluciones basadas en los hechos y reglas.

Ejercicio 14: Describa las características más importantes de los Lenguajes Basados en Scripts. Mencione diferentes lenguajes que utilizan este concepto. ¿En general, qué tipificación utilizan?

Características principales:

- Son generalmente interpretados.
- Permiten desarrollo rápido.
- Se utilizan para automatización.

- Sirven como "glue languages" para integrar aplicaciones.
- Poseen alta funcionalidad para tareas específicas.
- Utilizan pocos mecanismos de declaración.
- Son ideales para scripting web y administración de sistemas.

Ejemplos: Python, JavaScript, PHP, Perl, Tcl, Ruby. En general utilizan tipificación dinámica, en donde los tipos se determinan en tiempos de ejecución. Excepciones como TypeScript ofrecen tipado estático opcional.

Ejercicio 15: ¿Existen otros paradigmas? Justifique la respuesta.

- Paradigma imperativo: basado en secuencias de comandos que modifican el estado (C, Pascal).
- Paradigma procedimental: organiza código en procedimientos (Fortran, Ada).
- Estructurado: usa estructuras de control para evitar código desorganizado (C).
- Programación dirigida por eventos: responde a eventos externos (JavaScript).
- Concurrente: maneja tareas simultáneas con sincronización (Go).
- Programación orientada a aspectos: separa responsabilidades transversales.
- Programación reactiva: responde a flujos de datos o eventos en tiempo real (Elm).

Estos surgen para resolver problemas específicos y pueden combinarse con otros paradigmas.